

UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN PRAKTIKUM
APLIKASI WEB DAN MOBILE

Semester Genap 2025/2026

**MODUL11– React Routing & Multi-Page App +
Integrasi API**

Nama Mahasiswa : Nanik Wijayanti
NIM : 23051430006
Kelas : A2
Tanggal Praktikum : 07 Mei 2026
Dosen Pengampu : Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 11 (Pertemuan 11)
Topik Utama	React Routing & Multi-Page App + Integrasi API
Sub-Topik	Single Page Application (SPA) Navigation, React Router DOM, dan Fetch API dalam React.
Durasi Praktikum	340 Menit
Tanggal Praktikum	07 Mei 2026
Nama Mahasiswa	Nanik Wijayanti
NIM	23051430006
Kelas	A2

B. TUJUAN PEMBELAJARAN

Tuliskan tujuan pembelajaran yang ingin dicapai pada modul ini, sesuai dengan yang tertera di Manual Praktikum.

1	Memahami konsep SPA (Single Page Application).
2	Mampu mengatur navigasi antar halaman tanpa reload browser menggunakan `react router-dom`.
3	Menerapkan `fetch` API di dalam `useEffect` untuk mengambil data eksternal.
4	Membuat struktur aplikasi multi-halaman (Dashboard, Inventori, Settings).

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	Console Browser	Browser Modern	Untuk melakukan pengujian logika JavaScript, debugging,

			serta menampilkan hasil eksekusi kode secara langsung selama praktikum
5	Live Server	Ekstensi VS Code	Ekstensi untuk menjalankan server lokal agar perubahan kode web dapat terlihat secara <i>real-time</i> di browser

D. DASAR TEORI

Tuliskan ringkasan konsep/teori yang menjadi landasan pada modul ini. Ditulis dengan bahasa sendiri, bukan salinan mentah dari modul. Minimal 3 paragraf.

Pada pertemuan ke-11 ini mengeksplorasi konsep Single Page Application (SPA), sebuah pendekatan di mana aplikasi web terasa sangat responsif karena navigasi antar menu tidak memerlukan pemuatan ulang halaman secara keseluruhan. Landasan teknisnya terletak pada penggunaan pustaka React Router DOM yang berfungsi untuk mengelola alamat URL agar perpindahan dari satu bagian aplikasi ke bagian lain, misalnya dari pemantauan produksi ke manajemen inventori, terjadi secara instan tanpa gangguan visual yang biasanya ditemukan pada web statis konvensional.

Struktur aplikasi dikelola dengan memetakan setiap alamat URL ke komponen halaman tertentu melalui elemen Routes dan Route,. Hal yang paling krusial dalam mekanisme ini adalah penggunaan komponen Link untuk menggantikan tag tautan standar HTML. Komponen tersebut memastikan transisi antar modul tetap berada dalam lingkup SPA, sehingga data sementara yang sedang diproses tidak hilang dan aplikasi tidak mengalami *flicker* saat berpindah menu. Pengaturan ini membuat aplikasi yang kompleks tetap terorganisir dengan baik di dalam folder-folder halaman yang terpisah.

Aspek penting lainnya adalah Integrasi API untuk menghubungkan aplikasi dengan sumber data eksternal seperti database ERP atau sensor IoT,. Melalui pemanfaatan fungsi fetch yang dijalankan di dalam *hook* useEffect, aplikasi mampu menarik informasi dari server luar secara otomatis saat halaman dibuka. Bagi seorang insinyur industri, kemampuan ini sangat vital guna memastikan dashboard monitoring menampilkan data riil secara akurat dan tepat waktu, sehingga keputusan manajemen dapat diambil berdasarkan kondisi lapangan yang paling terkini.

D.1 Konsep Utama

- Penerapan Single Page Application (SPA)**
 Konsep ini memungkinkan aplikasi web berjalan sangat responsif karena navigasi antar menu tidak memicu pemuatan ulang halaman secara penuh (*full reload*). Dengan bantuan library React Router DOM, aplikasi tetap berada di satu halaman yang sama namun tampilannya berubah secara dinamis sesuai jalur URL yang diakses, sehingga performanya terasa cepat dan tidak ada gangguan visual (*flicker*) saat berpindah modul.
- Navigasi Tanpa Refresh dengan Komponen Link**
 Penggunaan komponen Link menjadi faktor kunci untuk menggantikan tag tautan HTML standar (<a>). Perbedaannya sangat krusial; komponen Link menjaga agar aplikasi tidak melakukan *refresh* total, yang artinya data atau

status (*state*) yang sedang diproses tidak akan hilang saat berpindah antar menu, misalnya dari halaman Dashboard utama ke halaman Inventori.

- **Struktur Folder dan Pemetaan Route**

Struktur aplikasi diatur secara modular dengan memisahkan kode ke dalam komponen Pages yang berbeda agar lebih mudah dirawat. Melalui elemen Routes dan Route, setiap alamat URL dipetakan ke komponen halaman tertentu secara spesifik, sehingga sistem tahu kapan harus menampilkan antarmuka pemantauan atau halaman error (404) jika alamat yang diakses tidak tersedia dalam sistem pabrik.

- **Integrasi Data Eksternal melalui API**

Aplikasi mampu terhubung dengan sumber data luar melalui Integrasi API menggunakan fungsi `fetch` yang dijalankan di dalam *hook* `useEffect`. Mekanisme asinkron ini memungkinkan aplikasi mengambil informasi secara otomatis dari database ERP perusahaan atau sensor IoT tepat saat halaman dibuka, sehingga data yang tersaji pada tabel riwayat atau inventori selalu bersifat akurat dan *real-time*

D.2 Keterkaitan dengan Teknik Industri

Pertemuan 11 ini memberikan fondasi penting dalam membangun sistem informasi industri yang tidak hanya fungsional tetapi juga efisien secara performa. Dengan beralih dari halaman web tradisional ke model Single Page Application (SPA), seorang insinyur dapat merancang antarmuka yang mendukung alur kerja cepat di lapangan tanpa hambatan teknis seperti pemuatan ulang halaman yang berulang. Hal ini memungkinkan integrasi berbagai modul operasional yang kompleks ke dalam satu kesatuan sistem yang sinkron dan responsif.

Berikut adalah beberapa poin keterkaitan dengan teknik industri:

- **Modularitas Sistem Informasi Industri yang Kompleks**

Di dunia industri, sistem manajemen seperti ERP (*Enterprise Resource Planning*) atau MES (*Manufacturing Execution System*) tidak mungkin hanya terdiri dari satu fungsi saja. Melalui penggunaan React Routing, berbagai modul yang berbeda seperti pemantauan produksi, manajemen inventori, dan pelaporan kualitas, dapat dipisahkan ke dalam halaman-halaman yang terorganisir namun tetap berada dalam satu aplikasi terpadu. Hal ini memudahkan pengaturan alur kerja operasional bagi pengguna di berbagai divisi pabrik tanpa tercampurnya data yang tidak relevan.

- **Peningkatan Efisiensi Kerja melalui Akses Cepat (SPA)**

Dalam lingkungan rantai produksi yang serba cepat, setiap detik sangatlah berharga untuk menghindari *bottleneck*. Konsep Single Page Application (SPA) memastikan perpindahan antar menu navigasi berjalan sangat instan tanpa harus memuat ulang halaman browser secara keseluruhan. Pengalaman pengguna yang mulus dan tanpa *flicker* ini sangat mendukung produktivitas operator saat harus berpindah cepat antara melakukan input data produksi dan mengecek status stok gudang.

- **Integrasi Data Real-Time untuk Pengambilan Keputusan**

Seorang insinyur industri sangat bergantung pada akurasi data untuk melakukan analisis masalah. Dengan menguasai Integrasi API, aplikasi web dapat dihubungkan langsung ke berbagai sumber data eksternal, mulai dari

basis data pusat perusahaan hingga sensor-sensor IoT di lini produksi. Hal ini menjamin bahwa informasi yang tersaji di layar dashboard, seperti jumlah stok bahan baku yang tersedia, merupakan data yang paling terkini (*real-time*), sehingga keputusan strategis dapat diambil dengan tingkat presisi yang lebih tinggi.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

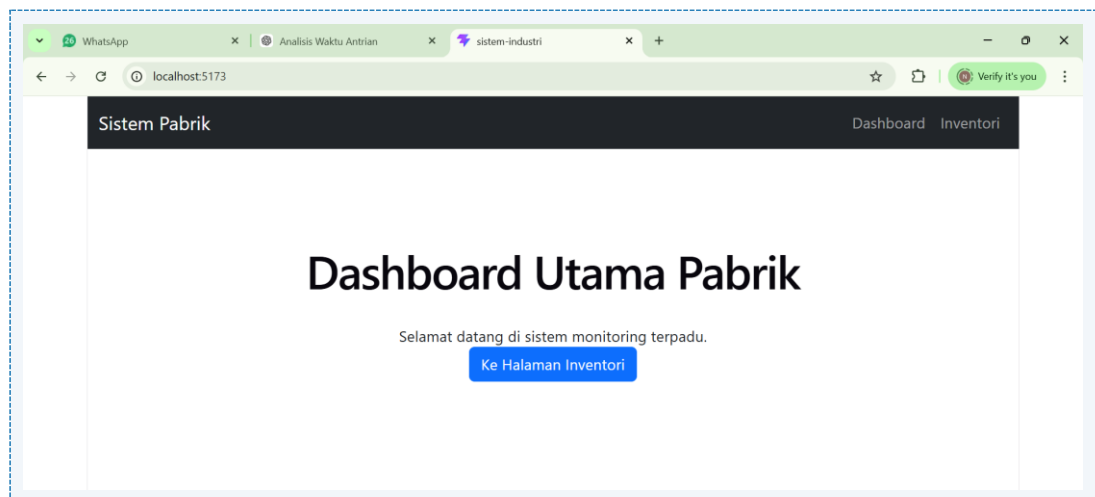
Dokumentasikan setiap langkah kerja yang Anda lakukan, beserta hasil yang diperoleh (screenshot, kode, atau output). Jawab sesuai urutan langkah di Manual Praktikum.

E.1 Langkah 1 – Setup Router

Buka file utama `src/main.jsx` (atau `index.jsx` tergantung versi Vite). Bungkus aplikasi dengan `BrowserRouter`.

```
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "../App.jsx";
import "../index.css";
ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>,
);
```

Screenshot Hasil:



Gambar E.1 – Tampilan Hasil Local Host

E.2 Langkah 2 – Membuat Halaman (Pages)

Buat folder `src/Halaman`.

Buat 3 file:

1. `Dashboard.jsx`
2. `Inventori.jsx`
3. `NotFound.jsx` (Halaman error).

Isi `src/Halaman/Dashboard.jsx`:

```
import React from "react";
import { Link } from "react-router-dom";
function Dashboard() {
  return (
    <div className="p-5">
      <h1>Dashboard Utama Pabrik</h1>
      <p>Selamat datang di sistem monitoring terpadu.</p>
      <Link to="/inventori" className="btn btn-primary">
        Ke Halaman Inventori
      </Link>
    </div>
  );
}
export default Dashboard;
```

Isi `src/Halaman/Inventori.jsx`:

```
import React, { useState, useEffect } from "react";
import { Link } from "react-router-dom";
function Inventori(){
  const [products, setProducts] = useState([]);
  // Latihan 2: State untuk loading (menandakan data sedang
  dimuat)
  const [loading, setLoading] = useState(true);
  // Fetch Data saat komponen mount
  useEffect(() => {
    setLoading(true); // [TAMBAHAN] aktifkan loading sebelum fetch

    fetch("https://jsonplaceholder.typicode.com/posts")
      .then((res) => res.json())
      .then((data) => {
        // Ambil hanya 5 data pertama untuk contoh
      })
  }, []);
}
```

```

        setProducts(data.slice(0, 5));
        setLoading(false); // Latihan 2: Mematikan loading setelah
data masuk
    })
    .catch((err) => {
        console.log(err);
        setLoading(false); // Latihan 2: mematikan loading kalau
error
    });
}, []);

return (
  <div className="container mt-4">
    <h1>Data Inventori Bahan Baku</h1>
    <Link to="/" className="btn btn-secondary mb-3">
      Kembali ke Dashboard
    </Link>
    {loading ? (
      <p className="text-center">Memuat data...</p>
    ) : (
      <table className="table table-striped">
        <thead>
          <tr>
            <th>ID Item</th>
            <th>Nama Bahan</th>
            <th>Status Supplier</th>
          </tr>
        </thead>
        <tbody>
          {products.map((item) => (
            <tr key={item.id}>
              <td>{item.id}</td>
              <td>
                {/* Link ke halaman detail menggunakan dynamic
route berdasarkan ID */}
                <Link
to={` /Inventori/${item.id}`}>{item.title}</Link>
              </td>{" "}
              {/* Menggunakan title sebagai simulasi nama bahan
*/}
              <td>
                <span className="badge bg-
success">Available</span>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    )}
  </div>
);

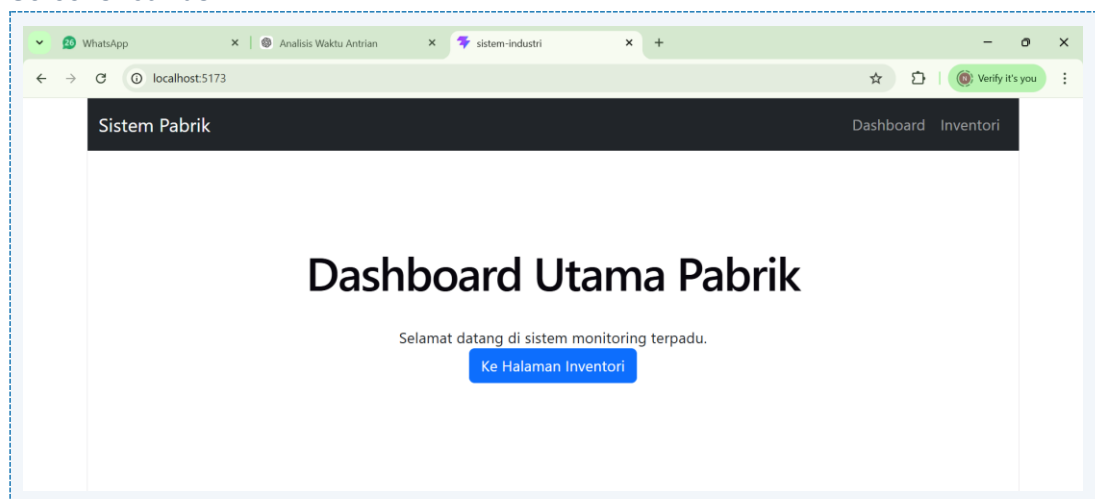
```

```
        </tbody>
      </table>
    )}
  </div>
);
}
export default Inventori;
```

Isi `src/Halaman/NotFound.jsx`:

```
import React from "react";
import { Link } from "react-router-dom";
function NotFound() {
  return (
    <div className="text-center mt-5">
      <h1 className="display-1">404</h1>
      <p>Halaman tidak ditemukan dalam sistem pabrik.</p>
      <Link to="/" className="btn btn-dark">
        Kembali ke Home
      </Link>
    </div>
  );
}
export default NotFound;
```

Screenshot Hasil:



Gambar E.2 – Tampilan Hasil Local Host

E.3 Langkah 3 – Mengatur Route di App.jsx

Sekarang kita tentukan kapan komponen mana yang muncul berdasarkan URL.

```
import React from "react";
import { Routes, Route } from "react-router-dom";
import Dashboard from "../Halaman/Dashboard";
import Inventori from "../Halaman/Inventori";
import NotFound from "../Halaman/NotFound";
// import Navbar from "../Komponen/Navbar"; // Anda bisa buat
// navbar sendiri
// Buat Navbar.jsx sederhana untuk navigasi
function Navbar() {
  return (
    <nav className="navbar navbar-expand-lg navbar-dark bg-dark
mb-4">
      <div className="container">
        <a className="navbar-brand" href="/">
          Sistem Pabrik
        </a>
        <div className="navbar-nav">
          <a className="nav-link" href="/">
            Dashboard
          </a>
          <a className="nav-link" href="/inventori">
            Inventori
          </a>
        </div>
      </div>
    </nav>
  );
}

function App() {
  return (
    <div>
      <Navbar />
      <Routes>
        {/* Route yang tepat akan di-render */}
        <Route path="/" element={<Dashboard />} />
        <Route path="/inventori" element={<Inventori />} />
        {/* Route untuk semua path lainnya (404) */}
        <Route path="" element={<NotFound />} />
      </Routes>
    </div>
  );
}

export default App;
```

Screenshot Hasil:

ID Item	Nama Bahan	Status Supplier
1	sunt aut facere repellat provident occaecati excepturi optio reprehenderit	Available
2	qui est esse	Available
3	ea molestias quasi exercitationem repellat qui ipsa sit aut	Available
4	eum et est occaecati	Available

Gambar E.3 – Tampilan Hasil Inventori

F. LATIHAN DAN TUGAS MANDIRI

Jawab dan dokumentasikan seluruh latihan/tugas yang ada di bagian "Latihan/Tugas Mandiri" pada modul Anda.

F.1 Latihan 1 – Dynamic Route

Pertanyaan / Instruksi Latihan:

Tambahkan Route untuk detail item. Misal path ``/inventori/:id``. Di halaman Inventori, ubah "Nama Bahan" menjadi Link ke halaman detail.

Jawaban / Kode Solusi:

Menambahkan kode di file `"Inventori.jsx"`

Menambahkan file baru yaitu halaman `DetailInventori.jsx`

```
import React from "react";
import { useParams, Link } from "react-router-dom";

function DetailInventori() {
  const { id } = useParams(); // ambil ID dari URL

  return (
    <div className="container mt-4">
      <h2>Detail Item Inventori</h2>
      <p>ID Item: {id}</p>

      <Link to="/Inventori" className="btn btn-secondary">
        Kembali ke Inventori
      </Link>
    </div>
  );
}
```

```
    </div>
  );
}

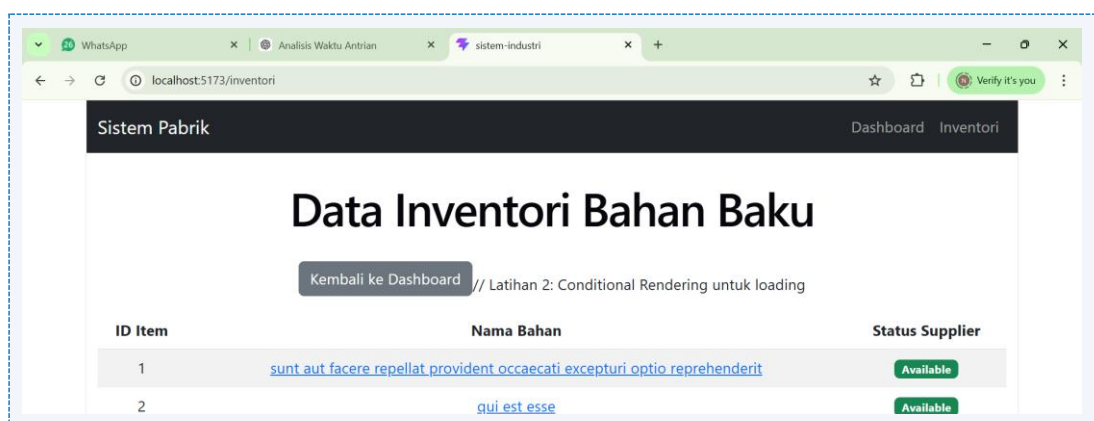
export default DetailInventori;
```

Mengubah kode sebelumnya menjadi seperti yang dibawah ini fi file halaman/Inventori.jsx

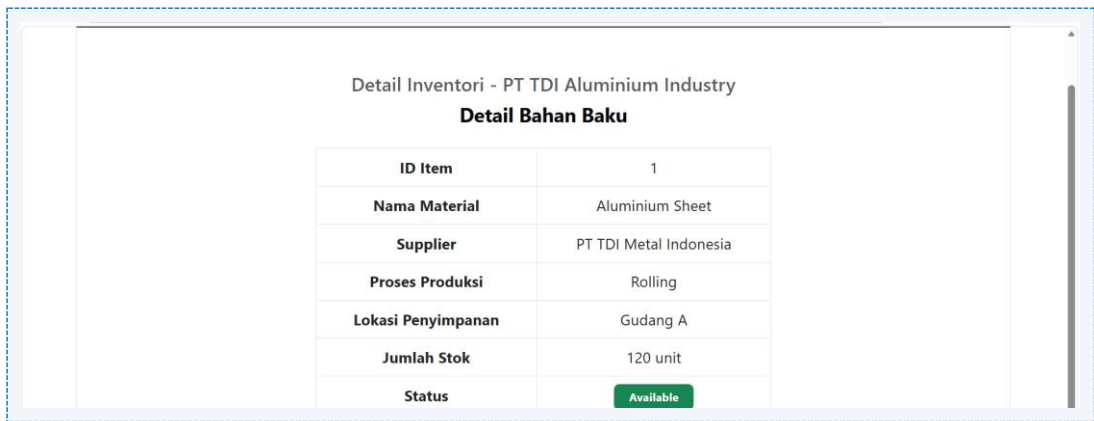
```
<td>
  { /* Link ke halaman detail menggunakan dynamic route
  berdasarkan ID */ }
  <Link to={` /Inventori/${item.id}`} >{item.title}</Link>
</td>{ " " }

// di App.jsx ditambahkan:
import DetailInventori from "../Halaman/DetailInventory"; //
Latihan 1: Menambahkan route untuk Detail Inventori

// Menambahkan route di App.jsx
<Route path="/Inventori" element={ <Inventori /> } />
  <Route path="/Inventori/:id" element={ <DetailInventori
/> } /> // Latihan 1: Menambahkan Route Detail Inventori
```



Gambar F.1 – Tampilan Hasil Latihan 1 dynamic Route



Detail Bahan Baku	
ID Item	1
Nama Material	Aluminium Sheet
Supplier	PT TDI Metal Indonesia
Proses Produksi	Rolling
Lokasi Penyimpanan	Gudang A
Jumlah Stok	120 unit
Status	Available

Gambar F.1 – Tampilan Hasil Detail inventory yang Link

-==

F.2 Latihan 2 – Loading State

Pertanyaan / Instruksi Latihan:

Di halaman `Inventori.jsx`, tambahkan state `loading` (boolean). Set `loading = true` sebelum fetch, dan `false` setelah data masuk. Tampilkan teks "Memuat data..." jika loading true.

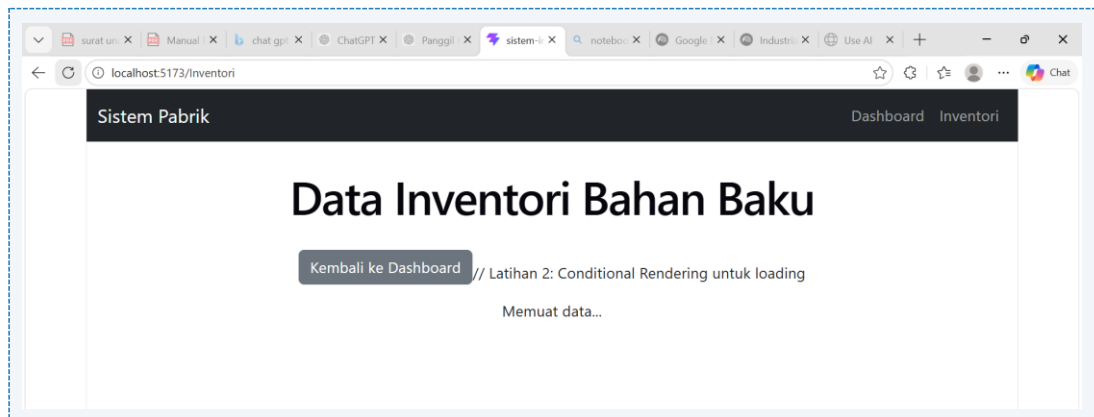
Jawaban / Kode Solusi:

Tambahkan kode pada halaman/Inventory.jsx:

```
// Latihan 2: State untuk loading (menandakan data sedang dimuat)
const [loading, setLoading] = useState(true);

.then((data) => {
  // Ambil hanya 5 data pertama untuk contoh
  setProducts(data.slice(0, 5));
  setLoading(false); // Latihan 2: Mematikan loading
  setelah data masuk
})
.catch((err) => {
  console.log(err);
  setLoading(false); // Latihan 2: mematikan loading kalau
  error
});
}, []);

// Latihan 2: Conditional Rendering untuk loading
{loading ? (
  <p className="text-center">Memuat data...</p>
) : (
```



Gambar F.2 – Tampilan Hasil Saat Memuat Data

F.3 Tugas Proyek Mini – Navigasi Modular

Deskripsi proyek mini:

Tugas Proyek Mini (Navigasi Modular):

- Buat halaman baru bernama `LaporanKualitas.jsx`. Halaman ini menampilkan data "Cacat" (mock data array manual).
- Tambahkan link di Navbar untuk menuju halaman ini.
- Pastikan transisi antar halaman berjalan mulus tanpa reload browser.

Penjelasan pendekatan/solusi yang digunakan:

[Tuliskan penjelasan Anda tentang cara menyelesaikan proyek mini ini]

Dalam menyelesaikan proyek mini Navigasi Modular ini, saya menggunakan konsep React Router untuk perpindahan halaman pada aplikasi web tanpa perlu melakukan reload browser. Langkah pertama yang saya lakukan adalah membuat halaman baru bernama "LaporanKualitas.jsx" yang digunakan untuk menampilkan data kualitas produksi aluminium. Pada halaman ini, saya menggunakan mock data berbentuk array of object yang berisi informasi produk, jenis cacat, serta jumlah cacat produksi.

Selanjutnya, saya menggunakan fungsi ".map()" untuk menampilkan seluruh data cacat ke dalam tabel secara dinamis. Dengan pendekatan ini, data menjadi lebih rapi, terstruktur, dan mudah dikelola. Saya juga menambahkan fungsi "reduce()" untuk menghitung total cacat produksi secara otomatis berdasarkan seluruh data yang ada pada array.

Selain itu, saya membuat fungsi "getStatus()" dan "getBadge()" untuk menentukan status kualitas produk berdasarkan jumlah cacat. Fungsi tersebut digunakan untuk menerapkan conditional rendering sehingga setiap status memiliki warna badge yang berbeda, misalnya hijau untuk kualitas baik, kuning untuk cukup, dan merah untuk kualitas buruk.

Dalam proses navigasi halaman, saya menggunakan react-router-dom dengan menambahkan Route baru pada App.jsx. Route tersebut menghubungkan URL /laporan-kualitas dengan komponen LaporanKualitas. Saya juga menambahkan menu navigasi pada Navbar atau Sidebar menggunakan komponen Link agar perpindahan halaman dapat berjalan secara mulus tanpa reload browser.

Selama pengerjaan proyek mini ini, saya sempat mengalami beberapa kendala seperti error import komponen, route yang tidak terbaca, serta halaman yang tidak muncul karena penempatan file yang kurang tepat. Untuk mengatasinya, saya melakukan pengecekan ulang pada struktur folder, memastikan nama file dan path import sudah sesuai, serta memperbaiki penempatan route pada App.jsx.

```
import React from "react";
import { Link } from "react-router-dom";

function LaporanKualitas() {
  // Data cacat industri aluminium (mock data)
  const dataCacat = [
    { id: 1, produk: "Aluminium Sheet", cacat: 6, jenis: "Goresan Permukaan" },
    {
      id: 2,
      produk: "Aluminium Profile",
      cacat: 3,
      jenis: "Dimensi Tidak Presisi",
    },
    {
      id: 3,
      produk: "Aluminium Foil",
      cacat: 8,
      jenis: "Ketebalan Tidak Seragam",
    },
  ];

  const totalCacat = dataCacat.reduce((total, item) => total + item.cacat, 0);

  const getStatus = (cacat) => {
    if (cacat >= 7) return "Buruk";
    if (cacat >= 4) return "Cukup";
    return "Baik";
  };

  const getBadge = (cacat) => {
    if (cacat >= 7) return "bg-danger";
    if (cacat >= 4) return "bg-warning text-dark";
    return "bg-success";
  };

  return (
```

```

<div className="container mt-4">
  <div className="text-center mb-4">
    <h2>Laporan Kualitas Produksi Aluminium</h2>
    <p className="text-muted">Nanik Wijayanti
(23051430006)</p>

    <h6 className="text-secondary">Proyek Mini: Navigasi
Modular</h6>
  </div>

  <Link to="/" className="btn btn-secondary mb-3">
    ← Kembali ke Dashboard
  </Link>

  <table className="table table-bordered text-center">
    <thead className="table-dark">
      <tr>
        <th>ID</th>
        <th>Produk</th>
        <th>Jenis Cacat</th>
        <th>Jumlah Cacat</th>
        <th>Status</th>
      </tr>
    </thead>
    <tbody>
      {dataCacat.map((item) => (
        <tr key={item.id}>
          <td>{item.id}</td>
          <td>{item.produk}</td>
          <td>{item.jenis}</td>
          <td>{item.cacat}</td>
          <td>
            <span className={`badge
${getBadge(item.cacat)}`}>
              {getStatus(item.cacat)}
            </span>
          </td>
        </tr>
      ))}
    </tbody>
  </table>

  <div className="alert alert-info text-center mt-3">
    Total Cacat Produksi: <strong>{totalCacat}</strong>
  </div>
</div>

```

```

    );
  }

  export default LaporanKualitas;

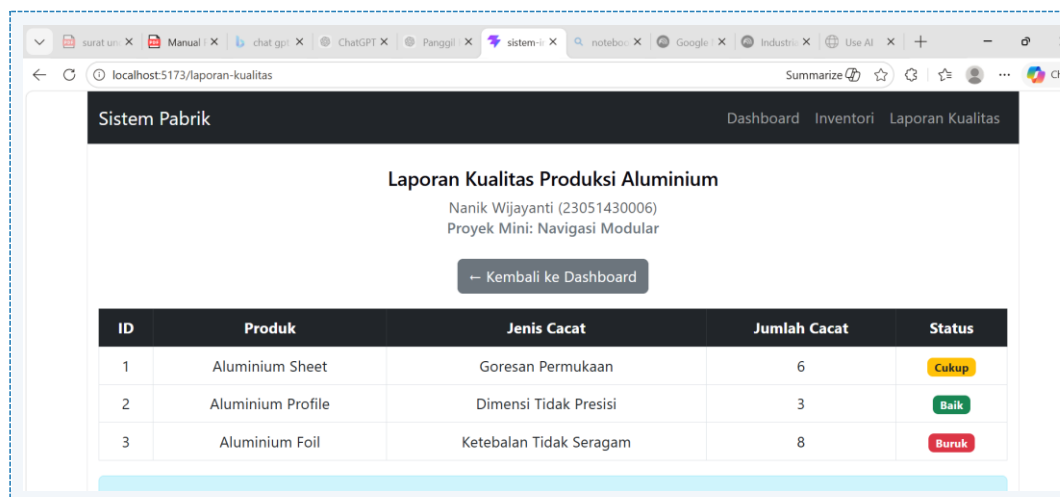
//Menambahkan import di App.jsx
import LaporanKualitas from "../Halaman/LaporanKualitas"; //Proyek
Mini: Menambahkan import laporan kualitas

//Menambahkan Route Laporan Kualitas
<Route path="/laporan-kualitas" element={<LaporanKualitas />} />
//
    Menambahkan route laporan kualitas 1: Menambahkan Route
Detail Inventori

// Menambahkan menu di Navbar

<a className="nav-link" href="/laporan-kualitas">
    Laporan Kualitas
  </a>

```



Gambar F.3 – Tampilan Hasil Laporan Kualitas

G. PEMBAHASAN

Analisis hasil praktikum secara kritis. Jawab pertanyaan berikut dengan lengkap dan mendalam (bukan hanya "ya/tidak").

G.1 Analisis Kesesuaian Hasil dengan Teori

Setelah menjalankan aplikasi dashboard monitoring produksi berbasis React, hasil yang diperoleh sudah cukup sesuai dengan teori yang dipelajari pada praktikum ini. Aplikasi berhasil menampilkan beberapa halaman seperti Dashboard, Inventori, Detail Inventori, dan Laporan Kualitas menggunakan sistem navigasi React Router. Perpindahan halaman dapat dilakukan tanpa reload browser sehingga aplikasi terasa lebih responsif dan interaktif. Hal ini sesuai dengan konsep Single Page Application (SPA) pada React, di mana perpindahan halaman dilakukan secara dinamis menggunakan route dan komponen.

Pada bagian pengolahan data, saya menggunakan `useState` untuk menyimpan data yang bersifat dinamis serta `useEffect` untuk menjalankan proses pengambilan data dari API ketika komponen pertama kali dibuka. Data inventori berhasil ditampilkan pada tabel secara otomatis setelah proses fetch selesai dilakukan. Selain itu, saya juga menambahkan loading state untuk menampilkan teks “Memuat data...” ketika data masih diproses. Hal ini sudah sesuai dengan teori asynchronous programming dalam React, di mana proses pengambilan data dilakukan secara asynchronous agar tampilan aplikasi tetap berjalan dengan baik.

Selanjutnya, pada fitur Dynamic Route saya menggunakan parameter URL seperti `/inventori/:id` untuk menampilkan detail item inventori berdasarkan ID tertentu. Dengan pendekatan ini, setiap item inventori dapat memiliki halaman detail sendiri tanpa harus membuat banyak halaman secara manual. Selain itu, saya juga menggunakan komponen `Link` dari `react-router-dom` agar perpindahan halaman dapat berjalan lebih mulus tanpa refresh browser. Hasil tersebut sudah sesuai dengan teori modular routing pada React yang bertujuan membuat aplikasi lebih terstruktur dan efisien.

Pada bagian dashboard visualisasi, saya menggunakan library `chart.js` dan `react-chartjs-2` untuk menampilkan grafik produksi harian serta grafik donat proporsi cacat produksi. Grafik berhasil menampilkan data produksi dan data cacat dalam bentuk visual yang lebih mudah dipahami dibandingkan tabel biasa. Selain itu, saya juga menerapkan conditional rendering untuk menampilkan warna status yang berbeda berdasarkan kondisi tertentu, misalnya status kualitas cacat dan status mesin produksi. Pendekatan ini sesuai dengan teori dashboard monitoring industri yang menekankan pentingnya visualisasi data agar informasi dapat dianalisis dengan lebih cepat.

Namun, secara kritis masih terdapat beberapa keterbatasan pada aplikasi yang dibuat. Sebagian data masih menggunakan mock data sehingga belum terhubung dengan database nyata dan belum sepenuhnya real-time. Selain itu, validasi input pengguna juga masih sederhana sehingga masih memungkinkan terjadinya input data yang kurang sesuai. Walaupun demikian, secara keseluruhan aplikasi ini sudah berhasil menerapkan konsep dasar React seperti component, state, routing, fetch API, conditional rendering, serta visualisasi data dashboard.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Aplikasi tidak muncul di localhost atau halaman kosong (blank page)	Saya memastikan server sudah dijalankan menggunakan <code>npm run dev</code> . Selain itu, saya mengecek Console (F12) untuk melihat error, seperti kesalahan import atau penulisan komponen yang tidak sesuai.

Error saat melakukan import komponen React	Saya mengecek kembali nama file, lokasi folder, serta penulisan path import agar sesuai dengan struktur project. Saya juga memastikan penggunaan huruf besar dan kecil sudah benar.
Sidebar atau Navbar tidak muncul pada halaman dashboard	Saya memastikan komponen Sidebar/Layout sudah dipanggil di dalam Dashboard.jsx dan memastikan posisi pemanggilan komponen berada di dalam return utama agar dapat dirender dengan benar.

G.3 Kaitan dengan Penerapan di Industri

Keterampilan pemrograman pada modul React Router dan navigasi modular sangat penting dalam pengembangan sistem informasi industri modern. Pada dunia industri, aplikasi tidak hanya terdiri dari satu halaman, tetapi memiliki banyak modul seperti dashboard produksi, inventori bahan baku, laporan kualitas, hingga monitoring mesin. Dengan menggunakan React Router, perpindahan antar halaman dapat dilakukan tanpa reload browser sehingga aplikasi menjadi lebih cepat, ringan, dan responsif saat digunakan operator maupun manajer produksi.

1. **Sistem Dashboard Produksi Terintegrasi**

Pada industri manufaktur seperti perusahaan aluminium, dashboard digunakan untuk memantau kondisi produksi secara real-time. Dengan konsep routing pada React, setiap modul dapat dipisahkan menjadi halaman tersendiri seperti Dashboard, Inventori, dan Laporan Kualitas. Hal ini memudahkan pengguna dalam mengakses informasi tertentu tanpa harus membuka aplikasi baru atau memuat ulang halaman. Sistem seperti ini banyak digunakan pada Manufacturing Execution System (MES) dan ERP industri modern.

2. **Monitoring Inventori Bahan Baku Secara Digital**

Halaman Inventori yang dibuat pada praktikum menggambarkan bagaimana data bahan baku dapat ditampilkan secara terstruktur menggunakan fetch API dan React Router. Dalam penerapan industri nyata, fitur ini digunakan untuk memantau stok aluminium, status supplier, maupun kebutuhan material produksi. Adanya dynamic route pada detail inventori juga membantu pengguna melihat informasi item tertentu secara lebih rinci, misalnya spesifikasi bahan, jumlah stok, atau kondisi penyimpanan.

3. **Laporan Kualitas dan Analisis Produk Cacat**

Halaman Laporan Kualitas memiliki kaitan langsung dengan proses Quality Control (QC) di industri. Data cacat seperti goresan permukaan, dimensi tidak presisi, dan ketebalan tidak seragam dapat digunakan untuk mengevaluasi performa produksi aluminium. Dengan tampilan tabel dan status kualitas, supervisor dapat lebih cepat mengidentifikasi masalah dominan pada proses produksi. Sistem digital seperti ini membantu perusahaan dalam mengurangi defect rate dan meningkatkan kualitas produk akhir.

4. **Navigasi Modular untuk Efisiensi Operasional**

Penggunaan navigasi modular memungkinkan aplikasi berkembang menjadi lebih besar tanpa membuat kode berantakan. Setiap halaman dipisahkan menjadi komponen yang memiliki fungsi masing-masing. Konsep ini sangat relevan di industri karena sistem manufaktur biasanya memiliki banyak divisi dan modul kerja. Dengan struktur modular, pengembangan aplikasi menjadi lebih mudah dipelihara, diperbaiki, dan dikembangkan sesuai kebutuhan perusahaan.

5. **Penerapan Industri 4.0 Berbasis Web Interaktif**

Secara keseluruhan, materi React Router dan navigasi modular mendukung penerapan konsep Industri 4.0 karena aplikasi dapat berjalan secara interaktif,

responsif, dan terintegrasi antar modul. Sistem tidak hanya menampilkan data, tetapi juga mempermudah proses monitoring, analisis, dan pengambilan keputusan secara cepat. Dengan teknologi web modern seperti React, perusahaan dapat membangun sistem monitoring produksi yang lebih fleksibel dan efisien dibandingkan sistem manual konvensional.

H. KESIMPULAN

Tuliskan 3–5 poin kesimpulan yang merangkum hal-hal yang dipelajari dan dicapai pada praktikum modul ini. Kesimpulan harus spesifik, bukan bersifat umum.

1	Saya memahami bahwa React Router sangat penting untuk membangun aplikasi web industri yang memiliki banyak halaman atau modul, seperti Dashboard, Inventori, dan Laporan Kualitas. Dengan routing, perpindahan halaman dapat dilakukan tanpa reload browser sehingga aplikasi terasa lebih cepat dan responsif saat digunakan.
2	Saya belajar cara membuat navigasi modular menggunakan komponen Route, Routes, dan Link dari React Router DOM. Dengan konsep ini, setiap halaman dapat dipisahkan menjadi file yang berbeda sehingga struktur program menjadi lebih rapi, mudah dikembangkan, dan mudah dipelihara.
3	Saya memahami penggunaan dynamic route untuk menampilkan detail data tertentu secara dinamis berdasarkan parameter URL. Melalui fitur ini, halaman detail inventori dapat menampilkan informasi item sesuai ID yang dipilih pengguna tanpa perlu membuat banyak halaman secara manual.
4	Saya belajar menggunakan fetch API dan useEffect untuk mengambil data secara otomatis dari sumber eksternal dan menampilkannya ke dalam tabel React. Selain itu, saya juga memahami pentingnya loading state agar pengguna mengetahui bahwa data sedang diproses sebelum berhasil ditampilkan.

I. DAFTAR PUSTAKA

Tuliskan semua sumber referensi yang digunakan dalam format APA 7th Edition. Minimal 3 referensi yang relevan (termasuk Manual Praktikum dan sumber online resmi seperti MDN, Reactjs.org, dll).

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
 Nanik Wijayanti NIM: 23051430006	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

PANDUAN DAN KETENTUAN PENGISIAN LAPORAN

Baca dengan seksama sebelum mengisi template laporan ini

1. Aturan Umum

a.	Laporan dikerjakan secara INDIVIDUAL, kecuali ada instruksi khusus dari dosen.
b.	Isi laporan harus merupakan hasil pekerjaan sendiri. Plagiarisme (menyalin laporan teman secara identik) akan mendapatkan nilai NOL (0) untuk kedua pihak.
c.	Bagian Dasar Teori HARUS ditulis dengan kalimat dan bahasa sendiri, bukan disalin mentah dari modul atau internet.
d.	Screenshot WAJIB memperlihatkan nama/NIM mahasiswa pada salah satu bagian tampilan (bisa di judul halaman web atau bagian lain) sebagai bukti keaslian pekerjaan.
e.	Semua teks dalam kotak yang berwarna abu-abu/biru muda dan bersifat miring adalah instruksi yang HARUS diganti dengan isi yang sesungguhnya.

2. Format File dan Ketentuan Pengumpulan

Hal	Ketentuan
Format File	Microsoft Word (.docx). Konversi ke PDF jika diminta dosen.
Nama File	Laporan_Modul[No]_[NIM]_[NamaLengkap].docx Contoh: Laporan_Modul02_12320001_AhmadDhani.docx
Ukuran Kertas	A4 (210 × 297 mm)
Margin	Atas: 3 cm Bawah: 3 cm Kiri: 4 cm Kanan: 3 cm
Font Teks Isi	Arial, ukuran 11pt, spasi 1.15
Font Kode Program	Courier New, ukuran 10pt, latar gelap (sudah tersedia di template)
Batas Pengumpulan	7 hari kalender setelah tanggal pelaksanaan praktikum
Media Pengumpulan	Github repository mahasiswa + konfirmasi email ke dosen
Subjek Email	[WebLab-No.Modul] NIM - Nama Lengkap Contoh: [WebLab-02] 12320001 - Ahmad Dhani

3. Kompilasi Laporan Akhir Semester

PENTING – LAPORAN AKHIR SEMESTER

Pada akhir semester, seluruh laporan modul (Modul 1–12, kecuali Modul 13 UTS dan Modul 16 UAS) akan dikompilasi menjadi SATU BERKAS LAPORAN AKHIR.
Ketentuan kompilasi:

a.	Laporan setiap modul dicetak/disimpan sebagai satu dokumen terpisah sesuai template ini.
b.	Kompilasi disatukan dengan urutan: Halaman Cover Akhir → Daftar Isi → Laporan Modul 1 → Laporan Modul 2 → ... → Laporan Modul 12 → Laporan Modul 14 & 15.
c.	Laporan UTS (Modul 13) dan UAS (Modul 16) dikumpulkan secara TERPISAH dengan ketentuan yang diumumkan saat ujian.
d.	Setiap laporan modul di dalam kompilasi diawali dengan halaman Cover (Bagian A di template ini) sebagai pemisah antar modul.
e.	Nama file laporan akhir: LaporanAkhir_[NIM]_[NamaLengkap].docx
f.	Laporan akhir dikumpulkan paling lambat H-3 sebelum UAS.

4. Ketentuan Pengumpulan via GitHub

```

PraktikumWebMob_TekInd_2026/
├── README.md
├── Laporan/
│   ├── Laporan_Modul01_[NIM].docx
│   ├── Laporan_Modul02_[NIM].docx
│   └── ... (dst hingga Modul 15)
├── Pertemuan01_[NIM]_[Nama]/
│   ├── index.html
│   └── style.css
├── Pertemuan02_[NIM]_[Nama]/
└── ... (dst)

```

Pastikan setiap Commit memiliki pesan yang deskriptif. Contoh: "Menambahkan Laporan Modul 02 – CSS Dasar". Lakukan Push ke GitHub setiap akhir praktikum.

5. Konsekuensi Keterlambatan Pengumpulan

Keterlambatan	Konsekuensi
1–2 hari	Pengurangan nilai 10% dari total skor laporan
3–4 hari	Pengurangan nilai 25% dari total skor laporan
5–7 hari	Pengurangan nilai 50% dari total skor laporan
> 7 hari	Laporan TIDAK DITERIMA (nilai 0) – wajib temui dosen untuk klarifikasi
Tidak dikumpulkan	Nilai laporan = 0, mahasiswa tidak diperkenankan mengikuti UAS

— Selamat mengerjakan —